

AKADEMIA TARNOWSKA
Katedra Informatyki · Informatyka — II stopień

Zaawansowany interfejs użytkownika

LABORATORIUM 10

Testowanie dostępności i audit WCAG

Czas trwania	Prowadzący	Efekty kształcenia
90 minut	mgr inż. Dominik Aksamit	IN2_K01 · IN2_K02 · IN2_W04

1. Cele zajęć

1.1. Powiązanie z efektami kształcenia

Laboratorium realizuje następujące efekty kształcenia określone w sylabusie przedmiotu:

Kod	Obszar	Cel operacyjny laboratorium
IN2_W04	Wiedza – metody oceny i testowania UI	Student zna i stosuje narzędzia do automatycznego i ręcznego audytu (Lighthouse, axe, pa11y, NVDA) oraz interpretuje raporty zgodności z WCAG 2.1 AA.
IN2_K01	Kompetencje – etyka i dostępność	Student rozumie prawne i etyczne aspekty dostępności cyfrowej oraz potrafi wdrażać rozwiązania zgodne ze standardami WCAG i EAA 2025.
IN2_K02	Kompetencje – komunikacja	Student sporządza raport z audytu zrozumiały dla programistów, grafików i interesariuszy projektu oraz potrafi merytorycznie odrzucić nieefektywne rozwiązania (np. accessibility overlays).

1.2. Cele szczegółowe

Po ukończeniu laboratorium student:

- przeprowadza pełny audyt dostępności za pomocą min. czterech narzędzi (GUI i CLI),
- identyfikuje i klasyfikuje błędy WCAG 2.1 według kryterium i poziomu (A / AA / AAA),
- sporządza raport z audytu z konkretnymi zaleceniami naprawy,
- implementuje poprawki w kodzie HTML/CSS dla 10 najczęstszych typów błędów,
- krytycznie ocenia zasadność stosowania accessibility overlays (argumenty techniczne, prawne, etyczne),
- rozumie pojęcie gotowości strony na agentów AI jako nowego wymiaru audytu frontendu.*

2. Dlaczego audit dostępności ma znaczenie — kontekst biznesowy

2.1. Audyty WCAG w projektach komercyjnych

Dostępność cyfrowa przestała być kwestią dobrej woli — stała się wymogiem prawnym, argumentem przetargowym i czynnikiem przewagi konkurencyjnej. Poniżej zestawienie kontekstów, w których audyty WCAG są dziś standardem branżowym.

Presales i przetargi publiczne

W postępowaniach przetargowych i ofertach dla dużych korporacji raport zgodności WCAG 2.1 AA bywa wymaganym załącznikiem. Firmy, które dostarczają go jako część oferty, dokumentują dojrzałość procesu wytwórczego i redukują ryzyko zamawiającego. To realny argument przewagi nad konkurencją bez tego procesu.

European Accessibility Act (EAA) — obowiązuje od 28 czerwca 2025

Dyrektywa UE 2019/882 (EAA) nakłada obowiązek zgodności z WCAG 2.1 AA na szeroki zakres produktów i usług cyfrowych: sklepy e-commerce, aplikacje bankowe, usługi streamingowe, terminale płatnicze, automaty biletowe. Brak zgodności grozi karami administracyjnymi i wykluczeniem z rynku unijnego.

Ważne: Komisja Europejska wprost stwierdziła: accessibility overlays (nakładki JS) nie stanowią właściwej drogi do zgodności z EAA — wymagana jest dostępność na poziomie kodu źródłowego.

Ryzyko prawne — USA i Europa

W USA: od 2024 roku ponad 1000 pozwów ADA/Sekcja 508 rocznie dotyczyło stron wyłącznie z nakładką overlay. Sądy konsekwentnie orzekają: symulowanie dostępności ≠ spełnianie standardów. FTC w 2025 roku ukarał lidera rynku overlayów karą 1 miliona dolarów za fałszywe twierdzenia o zgodności WCAG.

ROI dostępności

- Ponad 15% użytkowników ma stałą lub tymczasową niepełnosprawność wpływającą na korzystanie z internetu.
- Dostępny semantyczny HTML = lepszy SEO: alt-texty, hierarchia nagłówków, strukturalne dane.
- Koszt naprawy błędów dostępności po wdrożeniu jest 10–100× wyższy niż projektowanie dostępności od początku.
- Dostępność = dyscyplina kodowania — narzuca semantykę i strukturę, które polepszają maintainability.

2.2. Audit jako usługa — perspektywa rynkowa

Firmy świadczą audyty dostępności w kilku modelach:

1. Audit wstępny (pre-launch) — przed wdrożeniem, jako element procesu QA.
2. Audit certyfikacyjny — formalny dokument zgodności wymagany przez zamawiających i regulatorów.

3. Audyt ciągły (CI/CD) — automatyczne sprawdzanie przy każdym commitcie narzędziami CLI (pa11y, axe-core).
4. Audyt z udziałem użytkowników — testy z osobami z niepełnosprawnościami, badania eye-tracking.

3. Zasób dydaktyczny — strona HTML z celowymi błędami dostępności

Poniższy plik HTML zawiera 18 celowych błędów dostępności obejmujących różne kategorie kryteriów WCAG 2.1. Studenci otwierają plik lokalnie (przez serwer deweloperski) i przeprowadzają na nim pełny audyt zgodnie z instrukcją w sekcji 4.

Uwaga: Zapisz kod jako *bledy-dostepnosci.html* i uruchom: `npx serve .` (port 3000). Nie naprawiaj błędów przed audytem!

3.1. Wykaz 18 celowych błędów

#	Typ błędu	Kryterium WCAG 2.1	Poziom	Lokalizacja w kodzie
1	Brak atrybutu lang	3.1.1 Language of Page	A	<html> (bez lang)
2	Zbyt niski kontrast tekstu	1.4.3 Contrast (Minimum)	AA	.subtitle — #aaa/white (2.3:1)
3	Niewidoczny fokus (outline:none)	2.4.7 Focus Visible	AA	button, a, input
4	Niski kontrast na przycisku	1.4.3 Contrast (Minimum)	AA	.btn — #6699FF/#0043FF (1.6:1)
5	Pominięte poziomy nagłówków	1.3.1 Info and Relationships	A	<h1> → <h4> (brak h2, h3)
6	Brak atrybutu alt na obrazie	1.1.1 Non-text Content	A	
7	Pusty alt na obrazie inform.	1.1.1 Non-text Content	A	sale-banner.jpg alt=""
8	Linki nieopisowe	2.4.4 Link Purpose (In Context)	A	"Kliknij tutaj", "Więcej"
9	Input bez etykiety label	1.3.1 / 4.1.2 Name, Role	A	input#search (bez label)
10	Przycisk bez dostępnej nazwy	4.1.2 Name, Role, Value	A	<button>  </button>
11	Div klikalny bez klawiatury	2.1.1 Keyboard / 4.1.2	A	<div onclick> bez tabindex
12	Tabela bez nagłówków <th>	1.3.1 Info and Relationships	A	<table> (tylko <td>)
13	Video autoplay bez kontrolek	1.2.2 / 2.2.2 Pause, Stop	A	<video autoplay loop>
14	Pozytywne tabindex	2.4.3 Focus Order	A	tabindex="5", tabindex="1"
15	iframe bez tytułu	4.1.2 Name, Role, Value	A	<iframe> (bez title)
16	Radio bez fieldset/legend	1.3.1 Info and Relationships	A	<div><input radio...>

#	Typ błędu	Kryterium WCAG 2.1	Poziom	Lokalizacja w kodzie
17	Widget bez ról ARIA	4.1.2 Name, Role, Value	A	.dropdown (div onclick)
18	Ruchoma treść <marquee>	2.2.2 Pause, Stop, Hide	A	<marquee>PROMOCJA...</marquee>

3.2. Kod źródłowy strony demo

```
<!DOCTYPE html>
<!-- BŁĄD 1: Brak atrybutu lang na <html> -->
<html>
<head>
  <meta charset="UTF-8">
  <title>Sklep Demo - Strona główna</title>
  <style>
    body { font-family: Arial, sans-serif; background: #ffffff; }
    /* BŁĄD 2: Zbyt niski kontrast - #aaaaaa na białym tle (ratio ≈2.3:1) */
    .subtitle { color: #aaaaaa; font-size: 14px; }
    /* BŁĄD 3: Ukrycie wskaźnika fokus - brak widocznego fokus klawiatury */
    button, a, input { outline: none; }
    /* BŁĄD 4: Niski kontrast na przycisku (#6699FF na #0043FF ≈ 1.6:1) */
    .btn { background: #0043FF; color: #6699FF; padding: 10px 20px; border:
none; }
  </style>
</head>
<body>

  <!-- BŁĄD 5: Pominięte poziomy nagłówków - przejście z h1 bezpośrednio do h4 -
->
  <h1>Witaj w naszym sklepie</h1>
  <h4>Najnowsze produkty</h4>

  <!-- BŁĄD 6: Brak atrybutu alt na obrazie -->
  

  <!-- BŁĄD 7: Pusty alt="" na obrazie informatywnym (baner promocyjny) -->
  

  <!-- BŁĄD 8: Linki z nieopisowym tekstem -->
  <a href="/product/123">Kliknij tutaj</a>
  <a href="/product/456">Więcej</a>

  <!-- BŁĄD 9: Input bez powiązanego elementu <label> -->
  <input type="text" id="search" placeholder="Szukaj...">

  <!-- BŁĄD 10: Przycisk bez dostępnej nazwy (tylko emoji, brak tekstu/aria-
label) -->
  <button class="btn">&#128269;</button>

  <!-- BŁĄD 11: Klikalny <div> - nieosiągalny klawiaturą, brak roli -->
  <div onclick="addToCart()" style="cursor:pointer; background:#eee;
padding:10px;">
    Dodaj do koszyka
  </div>
```

```
<!-- BŁĄD 12: Tabela danych bez nagłówków <th> i scope -->
<table border="1">
  <tr><td>Produkt</td><td>Cena</td><td>Dostępność</td></tr>
  <tr><td>Laptop X</td><td>3499 zł</td><td>Tak</td></tr>
  <tr><td>Mysz Y</td><td>149 zł</td><td>Nie</td></tr>
</table>

<!-- BŁĄD 13: Video z autoplay i loop, bez kontrolek i napisów -->
<video src="promo.mp4" autoplay loop width="600"></video>

<!-- BŁĄD 14: Pozytywne wartości tabindex zaburzają naturalną kolejność fokus -->
<button tabindex="5">Kup teraz</button>
<button tabindex="1">Przejdź do kasy</button>

<!-- BŁĄD 15: iframe bez atrybutu title -->
<iframe src="https://maps.example.com/embed" width="600"
height="400"></iframe>

<!-- BŁĄD 16: Grupa radio bez <fieldset> i <legend> -->
<div>
  <p>Metoda dostawy:</p>
  <input type="radio" name="delivery" id="r1">
  <label for="r1">Kurier</label>
  <input type="radio" name="delivery" id="r2">
  <label for="r2">Paczkomat</label>
</div>

<!-- BŁĄD 17: Niestandardowy widget dropdown bez ról ARIA i zarządzania
fokusem -->
<div class="dropdown" onclick="toggleDropdown()"
  style="border:1px solid #ccc; padding:8px; cursor:pointer;">
  Wybierz kategorię ▼
  <div class="dropdown-menu" style="display:none;">
    <div onclick="select(1)">Elektronika</div>
    <div onclick="select(2)">Odzież</div>
  </div>
</div>

<!-- BŁĄD 18: <marquee> - ruchoma treść bez możliwości zatrzymania przez
użytkownika -->
<marquee>PROMOCJA: -20% na wszystkie produkty! Tylko dziś!</marquee>

</body>
</html>
```

4. Instrukcja przeprowadzenia audytu dostępności

Audyt przeprowadzamy w czterech warstwach: automatyczny CLI, automatyczny GUI, semi-automatyczny (screen reader) i ręczny. Żadna pojedyncza metoda nie wystarczy — każda wykrywa inne klasy błędów.

4.1. Lighthouse — GUI (Chrome DevTools)

Krok po kroku:

5. Otwórz stronę demo w Chrome. Naciśnij F12 (DevTools).
6. Przejdź do zakładki Lighthouse. Zaznacz kategorię Accessibility.
7. Wybierz tryb Navigation, kliknij Analyze page load. Poczekaj na wynik.
8. Kliknij przycisk ⋮ → Save as JSON (potrzebne do analizy CLI).

Interpretacja wyniku:

- Score 0–49 (czerwony): krytyczne problemy — strona praktycznie niedostępna.
- Score 50–89 (pomarańczowy): poważne braki wymagające naprawy przed wdrożeniem.
- Score 90–100 (zielony): zgodność z WCAG — utrzymuj w trakcie dalszego rozwoju.

Uwaga: Lighthouse używa silnika axe-core do wykrywania naruszeń WCAG. Wynik poniżej 80 na stronie demo jest celowy i oczekiwany.

4.2. Lighthouse — CLI (terminal)

Zalety CLI: integracja z pipeline CI/CD, raportowanie w JSON/HTML, audyt stron wymagających uwierzytelnienia, automatyczne blokowanie deploymentu przy regresji.

```
# Instalacja
npm install -g lighthouse

# Uruchom lokalny serwer (w katalogu z plikiem HTML)
npx serve .

# Audyt z raportem JSON (tylko kategoria accessibility)
lighthouse http://localhost:3000/blady-dostepnosci.html \
  --output json \
  --output-path ./raport-lighthouse.json \
  --only-categories=accessibility

# Raport HTML + JSON jednocześnie
lighthouse http://localhost:3000/blady-dostepnosci.html \
  --output html,json \
  --output-path ./raport

# Tryb headless (serwer CI, bez okna przeglądarki)
lighthouse http://localhost:3000 \
  --chrome-flags="--headless" \
```



```

--output json \
--output-path ./ci-report.json

# Odczyt score z JSON
cat raport-lighthouse.json | python3 -c \
    "import json,sys; d=json.load(sys.stdin); \
    print('Score:', int(d['categories']['accessibility']['score']*100)) "

```

4.3. axe DevTools — rozszerzenie GUI

Instalacja:

- Zainstaluj rozszerzenie axe DevTools ze sklepu Chrome Web Store (producent: Deque Systems).
- Po instalacji pojawia się nowa zakładka axe DevTools w Chrome DevTools (F12).

Przeprowadzenie audytu:

9. Otwórz stronę demo, naciśnij F12 → zakładka axe DevTools.
10. Kliknij Scan ALL of my page.
11. Przeglądaj wyniki sortowane wg. Impact: Critical → Serious → Moderate → Minor.
12. Filtruj wg. WCAG 2.1 AA — odfiltruj tylko naruszenia poziomu AA.
13. Kliknij Highlight, by zaznaczyć element na stronie. Kliknij Issue Description po link do dokumentacji WCAG.

4.4. axe-core — CLI

```

# Instalacja
npm install -g @axe-core/cli

# Audyt z tagiem WCAG 2.1 AA — wynik w terminalu
axe http://localhost:3000/blady-dostepnosci.html \
    --tags wcag2a,wcag2aa

# Eksport do JSON
axe http://localhost:3000/blady-dostepnosci.html \
    --tags wcag2aa \
    --reporter json > raport-axe.json

# Audyt wielu stron naraz
axe http://localhost:3000/page1.html \
    http://localhost:3000/page2.html \
    --tags wcag2aa

# Interpretacja wyjścia:
# Violations: 12    <- błędy wymagające naprawy
# Incomplete: 5    <- wymagają ręcznej weryfikacji (axe nie może ocenić
#                 automatycznie)
# Passes:         31    <- spełnione kryteria

```

4.5. pa11y — CLI do CI/CD

pa11y to lekkie narzędzie CLI zaprojektowane do integracji z pipeline'ami CI/CD (GitHub Actions, Jenkins, GitLab CI). Pozwala automatycznie blokować merge requesty, gdy strona nie spełnia progu dostępności.

```
# Instalacja
npm install -g pa11y pa11y-ci

# Podstawowy audyt (WCAG2AA)
pa11y http://localhost:3000/bledy-dostepnosci.html \
  --standard WCAG2AA

# Eksport do JSON
pa11y http://localhost:3000 --reporter json > pa11y-report.json

# Plik konfiguracyjny .pallyci.json (wiele stron):
{
  "defaults": { "standard": "WCAG2AA", "timeout": 30000 },
  "urls": [
    "http://localhost:3000/",
    "http://localhost:3000/product",
    "http://localhost:3000/checkout"
  ]
}

# Uruchomienie (exit code 1 gdy są błędy — blokuje CI)
pa11y-ci --config .pallyci.json

# Snippet GitHub Actions:
# steps:
#   - run: npx serve . &
#   - run: pa11y-ci --config .pallyci.json
```

4.6. NVDA / VoiceOver — screen reader i nawigacja klawiaturą

NVDA (Windows) — uruchomienie:

- Pobierz NVDA bezpłatnie ze strony [nvaccess.org](https://www.nvaccess.org).
- Uruchom NVDA (Ctrl+Alt+N) i otwórz stronę demo w Chrome lub Firefox.

Skrót NVDA	Co sprawdzamy
Tab	Kolejność fokus — czy jest logiczna? Czy fokus jest zawsze widoczny?
H	Nawigacja między nagłówkami — sprawdź hierarchię i kompletność
K	Lista linków — czy opisy są zrozumiałe poza kontekstem strony?
F	Następny formularz — czy pola mają etykiety odczytywane przez SR?
NVDA+F7	Lista elementów strony (nagłówki, linki, formularze) — przegląd struktury

VoiceOver (macOS) — kluczowe skróty:

- VO+U — Rotor (lista nagłówków, linków, formularzy, punktów orientacyjnych).

- VO+Cmd+H — następny nagłówek.
- Tab — nawigacja między elementami interaktywnymi.

4.7. Weryfikacja manualna

Kontrast kolorów:

14. Pobierz Colour Contrast Analyser (TPGi) lub użyj WebAIM Contrast Checker online.
15. Próbuj kolor tekstu i tła pipetą.
16. Minimalne ratio: 4,5:1 dla tekstu normalnego (WCAG 1.4.3 AA), 3:1 dla tekstu dużego ($\geq 18\text{pt}$ lub $\geq 14\text{pt}$ bold).

Kolejność fokus:

17. Zamknij mysz. Korzystaj wyłącznie z klawiatury (Tab, Shift+Tab, Enter, Spacja, strzałki).
18. Notuj kolejność elementów fokus i sprawdź: czy jest logiczna? Czy nie ma pułapek fokus?

Ważne: Pułapka fokus (focus trap): element, do którego można wejść Tabem, ale nie można go opuścić bez myszy.

5. Szablon raportu z auditu WCAG

Szablon wypełnia student podczas i po laboratorium. Każdy wykryty błąd opisuje w osobnym wierszu tabeli.

5.1. Dane ogólne auditu

Autor raportu:	_____
Data auditu:	_____
Audytowana strona / URL:	_____
Standard:	WCAG 2.1, Poziom AA
Narzędzia użyte:	Lighthouse / axe DevTools / pa11y / NVDA / Ręczna weryfikacja
Score Lighthouse (Accessibility):	___ / 100
Naruszenia axe-core (Violations):	Krytyczne: ___ Poważne: ___ Umiarkowane: ___ Drobne: ___

5.2. Tabela błędów

#	Błąd / Element	Kryterium WCAG	Poziom	Priorytet	Zalecenie naprawy	Status
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						

Uwaga: Priorytet: P1 = napraw przed wdrożeniem | P2 = napraw w ciągu 2 tygodni | P3 = zaplanuj w backlogu. Status: Otwarte / W trakcie / Naprawione / Odrzucone.

5.3. Podsumowanie i rekomendacje

Ogólna ocena dostępności strony:

3 najważniejsze rekomendacje:
1. 2. 3.

6. Instrukcja naprawy — Top 10 najczęstszych błędów

Błąd 1 — Brakujący atrybut alt

Kryterium: 1.1.1 Non-text Content (A). Wpływ: screen reader odczytuje nazwę pliku zamiast opisu treści.

PRZED (błąd)	PO (poprawka)
<pre></pre>	<pre><!-- Obraz informatywny – opisowy alt --> <!-- Obraz dekoracyjny – pusty alt --> </pre>

Błąd 2 — Brak etykiety dla pola formularza

Kryterium: 1.3.1 / 4.1.2 (A). Wpływ: screen reader nie przekazuje celu pola wejściowego.

PRZED (błąd)	PO (poprawka)
<pre><input type="text" id="search" placeholder="Szukaj..."></pre>	<pre><label for="search">Wyszukaj produkt</label> <input type="text" id="search" placeholder="Wpisz nazwę produktu"> <!-- Gdy etykieta niewidoczna: --> <input type="search" id="search" aria-label="Wyszukaj produkt" placeholder="Wpisz nazwę produktu"></pre>

Błąd 3 — Zbyt niski kontrast tekstu

Kryterium: 1.4.3 Contrast Minimum (AA). Wymaganie: 4,5:1 dla tekstu normalnego, 3:1 dla dużego.

PRZED (błąd)	PO (poprawka)
<pre>/* #aaaaaa na białym: ratio 2.3:1 – FAIL */ .subtitle { color: #aaaaaa; }</pre>	<pre>/* #767676 na białym: ratio 4.54:1 – PASS */ .subtitle { color: #767676; } /* Na szarym tle: */ .subtitle { color: #595959; background: #f5f5f5; }</pre>

Błąd 4 — Ukryty fokus klawiatury

Kryterium: 2.4.7 Focus Visible (AA). Wpływ: użytkownicy klawiatury nie widzą aktywnego elementu.

PRZED (błąd)	PO (poprawka)
--------------	---------------

```
/* Usuwa wskaźnik fokus – błąd krytyczny
*/
button, a, input { outline: none; }
```

```
/* Zachowaj fokus z własnym stylem */
:focus { outline: none; }
:focus-visible {
  outline: 3px solid #0043FF;
  outline-offset: 2px;
  border-radius: 2px;
}
```

Błąd 5 — Przycisk bez dostępnej nazwy

Kryterium: 4.1.2 Name, Role, Value (A). Wpływ: SR ogłasza "button" bez żadnego kontekstu.

PRZED (błąd)	PO (poprawka)
<pre><button class="btn">&#128269;</button></pre>	<pre><!-- Z widocznym tekstem: --> <button class="btn"> &#128269; Szukaj </button> <!-- Tylko ikona: --> <button class="btn" aria-label="Wyszukaj produkty"> &#128269; </button></pre>

Błąd 6 — Element <div> klikalny bez klawiatury

Kryterium: 2.1.1 Keyboard (A). Wpływ: użytkownicy klawiatury i SR nie mogą aktywować elementu.

PRZED (błąd)	PO (poprawka)
<pre><div onclick="addToCart()" style="cursor:pointer"> Dodaj do koszyka </div></pre>	<pre><!-- Najlepiej: użyj semantycznego <button> --> <button type="button" onclick="addToCart()"> Dodaj do koszyka </button> <!-- Gdy musisz użyć div: --> <div role="button" tabindex="0" onclick="addToCart()" onkeydown="if(event.key==='Enter' event.key===' ')addToCart()"> Dodaj do koszyka </div></pre>

Błąd 7 — Tabela bez nagłówek <th>

Kryterium: 1.3.1 Info and Relationships (A). Wpływ: SR nie powiązuje danych z nagłówkami kolumn.

PRZED (błąd)	PO (poprawka)
<pre><table border="1"> <tr> <td>Produkt</td> <td>Cena</td> </tr> <tr><td>Laptop</td><td>3499 zł</td></tr></pre>	<pre><table> <caption>Lista produktów z cenami</caption> <thead> <tr> <th scope="col">Produkt</th></pre>

<code></table></code>	<pre> <th scope="col">Cena</th> </tr> </thead> <tbody> <tr><td>Laptop</td><td>3499 zł</td></tr> </tbody> </table> </pre>
-----------------------------	--

Błąd 8 — iframe bez tytułu

Kryterium: 4.1.2 Name, Role, Value (A). Wpływ: SR nie informuje o zawartości ramki.

PRZED (błąd)	PO (poprawka)
<pre> <iframe src="map.html" width="600" height="400"> </iframe> </pre>	<pre> <iframe src="map.html" width="600" height="400" title="Mapa lokalizacji sklepu"> </iframe> </pre>

Błąd 9 — Grupa radio bez fieldset i legend

Kryterium: 1.3.1 Info and Relationships (A). Wpływ: SR nie odczytuje pytania przy każdym przycisku radio.

PRZED (błąd)	PO (poprawka)
<pre> <div> <p>Metoda dostawy:</p> <input type="radio" name="d" id="r1"> <label for="r1">Kurier</label> </div> </pre>	<pre> <fieldset> <legend>Wybierz metodę dostawy</legend> <div> <input type="radio" name="d" id="r1"> <label for="r1">Kurier</label> </div> <div> <input type="radio" name="d" id="r2"> <label for="r2">Paczkomat</label> </div> </fieldset> </pre>

Błąd 10 — Brak atrybutu lang na <html>

Kryterium: 3.1.1 Language of Page (A). Wpływ: SR używa błędnej wymowy lub głosu językowego.

PRZED (błąd)	PO (poprawka)
<pre> <!DOCTYPE html> <html> <!-- brak atrybutu lang --> </pre>	<pre> <!DOCTYPE html> <html lang="pl"> <!-- Angielska wersja strony: lang="en" --> <!-- Fragment innego języka w tekście: --> User Interface </pre>

7. Zadanie laboratoryjne

7.1. Treść zadania

Przeprowadź pełny audit dostępności swojego projektu z poprzednich laboratoriów (lub strony wskazanej przez prowadzącego) i sporządź raport z wynikami, używając szablonu z sekcji 5.

7.2. Wymagania dotyczące raportu

19. Uruchom Lighthouse CLI i zapisz raport JSON.
20. Uruchom axe CLI z tagiem wcag2aa i zapisz wyniki do JSON.
21. Uruchom pa11y z konfiguracją WCAG2AA i zapisz wyniki.
22. Przetestuj stronę czytnikiem ekranu NVDA lub VoiceOver — opisz min. 3 obserwacje.
23. Ręcznie zweryfikuj kontrast min. 5 elementów tekstowych narzędziem Colour Contrast Analyser.
24. Wypełnij szablon raportu z sekcji 5 — min. 10 opisanych błędów ze wszystkich narzędzi.
25. Napraw min. 5 wykrytych błędów i udokumentuj zmiany kodem przed/po.
26. Napisz podsumowanie: ogólna ocena dostępności + 3 najważniejsze rekomendacje.

7.3. Forma i termin oddania

- Raport w formacie PDF lub DOCX — używając szablonu z sekcji 5.
- Pliki JSON z raportów Lighthouse i axe dołączone jako załączniki.

8. Kryteria oceny raportu z audytu

Kryterium	Opis wymagania
Kompletność audytu	Użyto min. 3 narzędzi. Każde ma zapis wynikowy (JSON/HTML/zrzut ekranu).
Klasyfikacja błędów	Każdy błąd ma prawidłowo przypisane kryterium WCAG i poziom (A/AA/AAA).
Jakość zaleceń naprawy	Zalecenia konkretne z przykładem kodu. Nie ogólnikowe ("napraw kontrast").
Implementacja poprawek	Min. 5 błędów naprawionych z dokumentacją przed/po w kodzie.
Test screen readerem	Raport zawiera min. 3 obserwacje z testów NVDA/VoiceOver.
Podsumowanie i wnioski	Spójna ocena ogólna + priorytety naprawy + wnioski dotyczące procesu.

Material dodatkowy A — Accessibility Overlays: dlaczego NIE działają

Ważne: WIEDZA KRYTYCZNA: Klienci bardzo często kupują rozwiązania overlay jako "quick fix". Twój obowiązek jako specjalisty UI to umieć merytorycznie odrzucić tę propozycję i zaproponować właściwe podejście.

A.1. Czym jest accessibility overlay?

Accessibility overlay to rozwiązanie JavaScript wstrzykiwane do strony jako zewnętrzny skrypt. Po załadowaniu wyświetla pływający przycisk (często ikonę wózka inwalidzkiego). Po kliknięciu otwiera panel z opcjami: zwiększenie czcionki, wysoki kontrast, tryb dla dyslektyków, itp.

Popularne produkty: accessiBe, UserWay, AudioEye, EqualWeb, Allyable. Ceny: 500–2000 USD/rok. Obietnica: "pełna zgodność WCAG" bez dotykania kodu źródłowego.

A.2. Dlaczego NIE działają — argumenty techniczne

1. Nakrywa błędy, nie naprawia kodu

Overlay działa w przeglądarce użytkownika — nie zmienia kodu źródłowego. Gdy audytor sprawdza HTML, błędy są nadal obecne. Sądy w USA konsekwentnie orzekają: symulowanie dostępności ≠ spełnianie standardów.

2. Ograniczony zasięg wykrywania

Narzędzia automatyczne (na których opierają się overlaye) wykrywają ok. 30–40% naruszeń WCAG (dane W3C). Pozostałe 60–70% — złożone interakcje, kolejność fokus, jakość opisów obrazów, napisy wideo — wymaga oceny człowieka. Overlay nie może tego naprawić.

3. Aktywnie szkodzą użytkownikom screen readerów

- Zduplikowane anonsy: screen reader odczytuje ten sam tekst dwukrotnie — raz z HTML, raz z overlaya.
- Pułapki fokus: klawiatura wchodzi do panelu overlaya i nie może go opuścić.
- Nadpisywanie ustawień systemowych: overlay zmienia kontrast, który użytkownik już skonfigurował na poziomie OS.
- Wiele osób korzystających z technologii asystywnych aktywnie wyłącza overlaye jako utrudniające.

4. Ryzyko prawne — overlay nie chroni przed pozwem

USA (2024–2025): ponad 1000 pozwów ADA rocznie dotyczyło stron z zainstalowanym overlayem. FTC w 2025 roku ukarał lidera rynku overlayów (accessiBe) karą 1 miliona dolarów za wprowadzające w błąd twierdzenia o zgodności z WCAG.

5. Ryzyko bezpieczeństwa

Overlay to zewnętrzny skrypt ładowany z serwera dostawcy na każdej podstronie — poważne rozszerzenie powierzchni ataku. W 2022 roku w widżecie EqualWeb wykryto lukę DOM XSS dotykającą klientów takich jak Fiverr, Bosch i Avis.

6. EAA 2025 — overlay nie spełnia wymogów

European Accessibility Act wymaga, by treść była dostępna — nie tylko po uruchomieniu nakładki. Komisja Europejska wprost stwierdziła: overlay nie jest ścieżką do zgodności z EAA.

A.3. Jak argumentować z klientem?

27. Zapytaj o cel: "Czy chcecie uniknąć pozwu, czy zapewnić realną dostępność?" — to dwa różne cele.
28. Pokaż liczby: overlay adresuje 20–30% kryteriów. Raport axe/Lighthouse pokaże pozostałe.
29. Zaproponuj alternatywę: audyt + backlog napraw + dostępność wbudowana w proces projektowy od początku.
30. Wycenij: naprawa błędów dostępności we wczesnej fazie kosztuje 10–100× mniej niż po wdrożeniu.
31. Argument biznesowy: dostępny kod = lepszy SEO, niższy bounce rate, szersza grupa odbiorców.

Material dodatkowy B — isitagentready.com: audyt gotowości na agentów AI

Tradycyjny audyt WCAG ocenia dostępność dla ludzi. W erze agentów AI pojawia się nowy wymiar: czy strona jest czytelna i użyteczna dla autonomicznych agentów AI? Narzędzie isitagentready.com (Cloudflare, 2025) wypełnia tę lukę.

B.1. Co to jest isitagentready.com?

Narzędzie stworzone przez Cloudflare, opisywane jako "PageSpeed Insights, ale dla agentów AI zamiast dla prędkości strony". Nadaje stronie score 0–100 w pięciu kategoriach i generuje konkretne zalecenia. Mniej niż 5% stron jest dziś zoptymalizowanych pod kątem agentów AI (dane Cloudflare 2025).

Kontekst: web uczył się komunikować z przeglądarkami, potem z wyszukiwarkami (SEO). Teraz musi nauczyć się komunikować z agentami AI — Claude, ChatGPT, Perplexity — które przeglądają strony i wykonują zadania w imieniu użytkowników. Pojawiło się pojęcie AEO (Answer Engine Optimization) jako następcą SEO.

B.2. Co ocenia narzędzie?

Kategoria	Co sprawdza
Discoverability	Czy plik llms.txt istnieje? Czy zawiera semantyczne opisy stron z bogatym kontekstem i małą liczbą tokenów?
Content Accessibility	Czy treść jest w czystym HTML/Markdown czytelny dla LLM? Czy nie jest zablokowana JavaScript-em lub paywallem?
Bot Access Control	Czy robots.txt pozwala lub ogranicza boty AI (ClaudeBot, GPTBot)? Czy nagłówek Content-Signal jest ustawiony?
API / MCP Exposure	Czy strona eksponuje API lub narzędzia MCP dla agentów? Czy agent może wykonywać akcje na stronie?
Authentication Signals	Czy strona informuje agenta jak się uwierzytelnić? Czy nagłówki Link wskazują zasoby agentowe?

B.3. Plik llms.txt — kluczowy artefakt

Analogicznie do sitemap.xml dla wyszukiwarek, llms.txt to plik tekstowy w katalogu głównym strony. Zawiera ustrukturyzowaną listę podstron z semantycznymi opisami, pozwalając agentowi AI efektywnie znaleźć właściwą stronę bez kosztownych iteracyjnych poszukiwań.

```
# Przykład: https://example.com/llms.txt

# Dokumentacja API: https://example.com/docs
> Pełna dokumentacja techniczna API i SDK — dla developerów

## Produkty
- [Cennik] (https://example.com/pricing): Zestawienie planów: Free, Pro, Enterprise
```

```
- [Funkcje] (https://example.com/features): Szczegółowy opis wszystkich możliwości

## Wsparcie techniczne
- [FAQ] (https://example.com/faq): Najczęstsze pytania i odpowiedzi
- [Status] (https://example.com/status): Dostępność usług w czasie rzeczywistym
```

B.4. Związek z audytem frontendu

Agent readiness uzupełnia, nie zastępuje, audit WCAG. Oba wymiary są istotne: dostępność dla ludzi (WCAG) i dostępność dla agentów AI (llms.txt, czysty HTML, MCP). Strona może mieć Lighthouse score 100 i być zupełnie nieczytelna dla agentów AI.

Uwaga: Narzędzie bezpłatne pod adresem *isitagentready.com*. W trybie zaawansowanym wybierz typ: *marketing website* vs. *aplikacja* — logika oceny jest różna dla obu przypadków.

Material dodatkowy C — WebMCP w kontekście audytu frontendu

C.1. Czym jest WebMCP?

WebMCP (Web Model Context Protocol) to proponowany standard webowy rozwijany przez Google i Microsoft w ramach grupy W3C Web Machine Learning. Wprowadza nowe API przeglądarki: `navigator.modelContext` — każda strona WWW może deklarować własne narzędzia dostępne dla agentów AI działających w przeglądarce.

```
// Przykład: strona rejestruje narzędzie dla agentów AI
// (kod po stronie właściciela serwisu)

navigator.modelContext.register({
  name: "search_products",
  description: "Wyszukuje produkty w katalogu sklepu",
  parameters: {
    query: { type: "string", description: "Fraza wyszukiwania" },
    category: { type: "string", enum: ["elektronika", "odzież", "sport"] }
  }
});

// Status: Chrome 146+ Canary
// Flaga: chrome://flags/#enable-webmcp-testing
// Standard: W3C Web ML Community Group (Google + Microsoft)
```

C.2. WebMCP vs. screen scraping — zmiana paradygmatu

Tradycyjnie agenci AI przeglądający strony musieli parsować DOM lub zrzuty ekranu (screen scraping). WebMCP zastępuje to strukturalnym, wysokowydajnym interfejsem. Demonstracja z grudnia 2025 roku pokazała 90% redukcję zużycia tokenów przy nawigacji stron z WebMCP w porównaniu do podejścia bez WebMCP.

C.3. Trzy poziomy dostępności frontendu

Nowoczesny interfejs musi być czytelny na trzech poziomach:

- Dla człowieka — klasyczna użyteczność i dostępność (WCAG).
- Dla screen readera / technologii asystywnych — semantyczny HTML, ARIA, poprawna hierarchia.
- Dla agentów AI — WebMCP, `llms.txt`, czysty DOM, strukturalne dane, MCP exposure.

Oznacza to, że audyt nowoczesnego frontendu powinien obejmować pytanie: "Czy agent AI może wykonać na tej stronie typowe zadania użytkownika?"

C.4. WebMCP DevTools — narzędzie inspektora

WebMCP DevTools to rozszerzenie Chrome (Chrome Web Store) do inspekcji, testowania i monitorowania narzędzi WebMCP zarejestrowanych przez stronę.

- Automatyczne wykrywanie wszystkich narzędzi WebMCP na bieżącej stronie.
- Wizualizacja schematów wejścia jako drzewo JSON Schema.
- Interaktywne formularze generowane ze schematu — testowanie bez pisania kodu.
- Historia ostatnich 20 wywołań z danymi wejścia i wyjścia.
- Eksport definicji narzędzi jako JSON, Markdown, kolekcja Postman lub JavaScript.

Uwaga: Jak włączyć: `chrome://flags/#enable-webmcp-testing` → *Enable*. Wymaga Chrome 146 Canary lub nowszego.

Material dodatkowy D — Chrome DevTools MCP: audyt frontendu z pomocą agenta AI

D.1. Czym jest Chrome DevTools MCP?

Oficjalny serwer MCP opublikowany przez Google Chrome (public preview wrzesień 2025). Umożliwia agentom AI (Claude Code, Cursor, GitHub Copilot, Gemini CLI) bezpośredni dostęp do Chrome DevTools: konsoli, sieci, drzewa dostępności, śladów wydajności.

Rozwiązujący problem: agenci AI piszący kod frontendowy "programują z zawiązanymi oczami" — nie widzą, co wygenerowany kod robi w przeglądarce. Chrome DevTools MCP to zmienia.

D.2. Instalacja i konfiguracja

```
# Instalacja i uruchomienie
npx chrome-devtools-mcp@latest

# Konfiguracja w Claude Code / Cursor (plik servers.json):
{
  "servers": {
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "chrome-devtools-mcp@latest"]
    }
  }
}

# Połączenie z uruchomioną instancją Chrome (np. w Docker):
npx chrome-devtools-mcp@latest --browser-url http://localhost:9222

# Tryb izolowany — bez dostępu do danych profilu użytkownika (zalecany do testów):
npx chrome-devtools-mcp@latest --isolated=true
```

D.3. Narzędzia MCP dostępne w audycie dostępności

Narzędzie MCP	Zastosowanie w audycie dostępności
take_snapshot	Tekstowa reprezentacja accessibility tree — agent widzi stronę tak jak screen reader: role, nazwy, stany. Wykrywa brakujące etykiety, role i relacje ARIA.
navigate_page	Otwiera stronę do audytu. Łączymy: "Nawiguj do strony i sprawdź czy formularz logowania jest dostępny."
press_key (Tab)	Symuluje wciśnięcia klawisza Tab — agent testuje kolejność fokus i wykrywa pułapki fokus automatycznie.
screenshot	Przechwytuje zrzut ekranu — Claude analizuje go wzrokowo: wskaźnik fokus, kontrast, układ elementów.

Narzędzie MCP	Zastosowanie w audycie dostępności
console_messages	Pobiera błędy i ostrzeżenia z konsoli — wykrywa błędy ARIA i nieprawidłowe atrybuty dostępności raportowane przez Chrome.
performance_trace	Rejestruje ślad wydajności — audyt może obejmować jednocześnie dostępność i Core Web Vitals.

D.4. Przykładowe prompty do audytu a11y przez agenta

```
# Prompt 1: Analiza accessibility tree
"Otwórz http://localhost:3000/bledy.html, zrób snapshot
accessibility tree i wymień wszystkie elementy bez dostępnej nazwy."

# Prompt 2: Test kolejności fokus
"Nawiguj do strony, wciśnij Tab 15 razy i opisz kolejność
elementów fokus. Czy kolejność jest logiczna?"

# Prompt 3: Wykryj błędy ARIA w konsoli
"Otwórz stronę, pobierz błędy konsoli i znajdź te
związane z ARIA lub dostępnością."

# Prompt 4: Napraw i zweryfikuj
"Znajdź wszystkie input bez label lub aria-label.
Napraw je w kodzie, a następnie zrób nowy snapshot
i potwierdź że poprawka działa."
```

D.5. BrowserTools MCP — alternatywa z pełnym Lighthouse

BrowserTools MCP (AgentDesk AI) to alternatywny serwer MCP integrujący Chrome Extension + Node Server + MCP. Oferuje polecenie run_accessibility_audit oparte na Lighthouse i axe-core. Agent może uruchomić pełny audyt WCAG bezpośrednio z poziomu promptu i otrzymać gotowy raport.

```
# Prompt dla BrowserTools MCP:
"Uruchom audyt dostępności na bieżącej stronie.
Podaj listę wszystkich naruszeń WCAG 2.1 AA posortowanych
wg. Impact: Critical, Serious, Moderate, Minor."
```

D.6. Podsumowanie — ekosystem narzędzi audytu 2025/2026

Narzędzie	Typ	Główne zastosowanie	Status
Lighthouse	GUI + CLI	Audyt a11y + wydajność + SEO	Produkcyjny
axe-core	GUI + CLI + JS API	Szczegółowy audyt WCAG	Produkcyjny
pa11y	CLI + CI/CD	Automatyczne testy w pipeline	Produkcyjny
NVDA / VoiceOver	Screen reader	Realne testy z technologiami asystyw.	Produkcyjny

Narzędzie	Typ	Główne zastosowanie	Status
Chrome DevTools MCP	MCP Server (oficjalny)	Audyt a11y przez agenta AI w Chrome	Public Preview
isitagentready.com	SaaS (Cloudflare)	Agent readiness score 0–100	Dostępny (2025)
WebMCP DevTools	Rozszerzenie Chrome	Inspekcja narzędzi WebMCP	Early Preview

Opracowanie: mgr inż. Dominik Aksamit · Katedra Informatyki · Akademia Tarnowska · rok akad. 2025/2026